Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
Deuxième Année Ingénieur Informatique
Matière : Algorithmiques et complexité

Solution Fiche de TD n° 02
Algorithmes de Tri

Solution exercice n° 01 :

La complexité de la fonction **find_min_max** pour trouver simultanément le minimum et le maximum parmi une liste de n entiers est de **O(n)** car la fonction parcourt la liste une seule fois, en comparant chaque entier avec les valeurs actuelles du minimum et du maximum. Ainsi, le nombre d'opérations de comparaison est directement lié à la taille de la liste (n), ce qui en fait une complexité linéaire **O(n).**

Solution exercice n° 02:

Ce programme calcul simultanément le plus grand et le deuxième plus grand élément de n entiers.

Compléxité : Nombre de comparaisons :

Le nombre de comparaisons dans l'algorithme pour trouver simultanément le plus grand et le deuxième plus grand élément dans une liste de n entiers :

Dans le pire des cas, l'algorithme effectue n - 1 comparaisons pour trouver le plus grand élément, et n - 2 comparaisons supplémentaires pour déterminer le deuxième plus grand élément, ce qui totalise 2n - 3 comparaisons. Donc, le nombre total de comparaisons dans le pire des cas est de n - 1 (pour le maximum) + n - 2 (pour le deuxième maximum) = 2n - 3 comparaisons. Cela rend l'algorithme O(n) en termes de complexité en nombre de comparaisons.

Exercice n° 03 :

```
def tri(T, p):
    if p > 1:
        for i in range(p - 1):
            if T[i] > T[i + 1]:
                T[i], T[i + 1] = T[i + 1], T[i]
        tri(T, p - 1)
T = [5.0, 3.0, 8.0, 7.0, 1.0]
tri(T, len(T))
print(T)
```

Le programme effectue un **tri à bulles** récursif sur le tableau T.

Il compare les éléments adjacents du tableau et les échange si le premier est plus grand que le second.

Il répète ce processus jusqu'à ce que le tableau soit trié par ordre croissant.

Voici le résultat de l'exécution du programme: T = [5.0, 3.0, 8.0, 7.0, 1.0]

tri(T, 5) ⇒ T = [3.0, 5.0, 7.0, 1.0, 8.0]

tri(T,4)
tri(T,4) ⇒ T = [3.0, 5.0, 1.0, 7.0, 8.0]
tri(T, 3)
tri(T, 3) ⇒ T = [3.0, 1.0, 5.0, 7.0, 8.0]
tri(T, 2)
tri(T, 2) ⇒ T = [1.0, 3.0, 5.0, 7.0, 8.0]
tri(T, 1)
tri(T, 1) ⇒ T = [1.0, 3.0, 5.0, 7.0, 8.0]

1.0, 3.0, 5.0, 7.0, 8.0

La complexité du programme :
Ce programme compare les éléments adjacents d'un tableau et les échange si nécessaire, puis rappelle la fonction tri sur le sous-tableau de taille p-1. Le tableau est trié lorsque p devient égal à 1. La complexité d'un algorithme est le nombre d'opérations élémentaires qu'il effectue en fonction de la taille des données en entrée.
Dans le cas du tri par bulles, les opérations élémentaires sont les comparaisons et les échanges entre les éléments du tableau. On peut compter le nombre d'opérations en fonction de la taille du tableau, notée n. Le programme effectue une boucle for de 0 à p-1, soit n-1 itérations, à chaque appel récursif de la fonction tri. À chaque itération, il effectue une comparaison et éventuellement un échange, soit deux opérations au maximum. Donc, le nombre d'opérations pour un appel récursif est au plus 2(n-1). Le programme appelle récursivement la fonction tri avec p-1 comme argument, jusqu'à ce que p soit égal à 1. Donc, le nombre d'appels récursifs est n. Ainsi, le nombre total d'opérations est au plus 2(n-1) + 2(n-2) + ... + 2(1) + 2(0), ce qui est égal à $2n(n-1)/2$, soit n^2-n opérations. Ainsi, la complexité du programme en notation big O est $O(n^2)$,

Exercice n° 04
La complexité temporelle de la fonction qui fusionne deux tableaux triés pour créer un nouveau tableau trié dépend de la longueur des deux tableaux, notée n et m, respectivement.

Dans un tel algorithme de fusion, le parcours des deux tableaux et la comparaison des éléments pour les fusionner dans le nouvel tableau. La complexité de cet algorithme de fusion est O(n + m), où n est la longueur du premier tableau et m est la longueur du deuxième tableau.

Exercice n°05 :
La complexité temporelle de l'algorithme qui trouve le plus grand produit de trois nombres d'un tableau dépend de la manière dont l'algorithme est implémenté.
N est la taille du tableau et la complexité temporelle est $O(N^3)$, tel qu'il est traité à travers des boucles triplement imbriquées. En réalité, la boucle du milieu s'exécute N/2 fois, et la boucle la plus interne s'exécute N/4 fois, c'est donc N * (N/2) * (N/4), ce qui est N^3 / 8 étapes. Mais nous supprimons la constante, ce qui nous laisse avec $O(N^3)$.

Voici deux approches améliorant la complexité précédente :

1. Tri du tableau : Une approche courante consiste à trier le tableau en ordre croissant (ou décroissant) en utilisant un algorithme de tri, comme le tri rapide ou le tri fusion, ce qui prend O(n log n) opérations dans le pire cas. Une fois que le tableau est trié, les trois plus grands éléments (qui sont situés à la fin du tableau) ou les deux plus petits éléments (qui sont situés au début du tableau) sont utilisés pour calculer le produit le plus élevé. L'étape de tri est la partie la plus coûteuse en termes de temps dans cet algorithme, donc la complexité temporelle totale est O(n log n).

2. Recherche directe : Une autre approche consiste à parcourir le tableau pour trouver les trois plus grands éléments et les deux plus petits éléments. Cela peut être fait en un seul passage du tableau en maintenant cinq variables pour suivre ces éléments tout en parcourant le tableau. La complexité temporelle de cette approche est O(n), car il s'agit d'une simple boucle sur le tableau.

Donc, en fonction de la méthode utilisée, la complexité temporelle pour trouver le plus grand produit de trois nombres d'un tableau donné peut être O(n log n) avec le tri suivi de la sélection des éléments, ou O(n) avec la recherche directe des éléments.